

The Ledger Valuation Stack

Pricing, Greeks, Calibration, PnL Attribution, and Temporal Orchestration

Companion Document to the Ledger Framework Specification

R. Delloye

March 2026 — v1.0

Abstract

The Ledger framework [1] defines portfolio value as $V_t = \sum_u \text{own}(u) \cdot P_t(u)$ and proves that PnL is path-independent. The framework deliberately treats P_t as an external input without specifying how that function is computed, validated, or delivered.

This companion document fills the gap. Version 1.0 makes five contributions beyond the v0.2 draft.

First, it introduces a *finite state machine* (FSM) for the valuation lifecycle of every unit. Eight states—UNPRICED, PRICING, PRICED, EXPLAINING, EXPLAINED, QUARANTINED, STALE, and FAILED—govern the progression from market data arrival through pricing, PnL explain verification, and publication. Every transition has a triggering event and a guard condition; terminal and absorbing behaviours are explicit.

Second, it formalises the *Kalman filter for market data calibration*, drawing from the Calibration Manifesto [5]. The state vector (latent curve or surface parameters), observation model (market observables as functions of state), transition dynamics (driftless random walk consistent with the martingale axiom), noise structure (bid-ask-derived observation noise and process noise governing adaptation speed), and the predict-update cycle are specified. Innovation-based gating rejects bad data; filtered estimates feed the pricing DAG as calibrated market data nodes.

Third, it reframes *Greeks through the parameter-martingale lens*: a model has parameters assumed to be martingales; Greeks are partial derivatives with respect to observables, holding parameters constant. The key insight is that “vega” only exists as a scalar when volatility is a single parameter (Black–Scholes); in multi-parameter models (Heston, SABR, local vol), vega is replaced by the *sensitivity Jacobian*—a vector whose dimension equals the number of model parameters. This framework is developed concretely: Black–Scholes delta versus local vol delta under a skewed surface, total gamma incorporating vanna, volga, and vega-curvature adjustments [3, 4], the toxic product boundary, the consensus protocol for multi-model disagreement, and practical computation methods (analytical, bump-and-revalue, AAD, pathwise/likelihood ratio).

Fourth, it tightens *integration with the Ledger*: the price function $P(u, \text{state}_t(u), \text{market_data}_t)$ is a formal definition, the lifecycle-valuation ordering protocol is embedded in the FSM, and the Taylor approximation is explicitly positioned as the price in state EXPLAINED between repricing cycles.

Fifth, it hardens the *production architecture* with a scalability analysis: one workflow per unit at 100,000 units with 30-second average cadence yields 200,000 workflow executions per minute, within Temporal cluster capacity with horizontal scaling.

The document retains all material from v0.2 (the ValuationRecord, Pricing DAG, Temporal workflow architecture, PnL explain gate, continuous approximate pricing, valuation store, edge cases, worked examples, and lifecycle consistency) and integrates the new contributions into the existing structure.

Contents

1 Introduction

4

1.1	Relationship to the Ledger Specification	4
1.2	Relationship to the Reference Papers	4
1.3	Document Roadmap	5
2	The Valuation Lifecycle FSM	6
2.1	States	6
2.2	Transitions	6
2.3	FSM Diagram	7
2.4	FSM and the Pricing Workflow	7
2.5	FSM and the Taylor Approximation	7
3	The Valuation Record	9
3.1	Quality Semantics	9
3.2	Dirty Price, Clean Price, and Accrued Interest	9
4	Greeks, Parameters, and the Sensitivity Jacobian	10
4.1	Models, Parameters, and the Martingale Assumption	10
4.2	Greeks as Parameter-Invariant Sensitivities	10
4.3	The Vanishing Vega Problem	11
4.4	The Sensitivity Jacobian	11
4.5	Why the Invariant Matters for PnL	12
4.6	Black–Scholes Delta vs. Local Vol Delta: A Concrete Comparison	12
4.7	Total Delta: Incorporating Empirical Parameter-Spot Dependence	13
4.8	Total Gamma Under Smile-Dependent Volatility	13
4.9	PnL Attribution with the Jacobian	14
4.10	The Toxic Product Boundary	15
4.11	The Consensus Problem: Multiple Models	15
4.12	Practical Computation of Greeks	15
4.13	Greeks Structures by Instrument Class	16
5	Kalman Filter for Market Data Calibration	19
5.1	State Vector	19
5.2	Transition Dynamics	19
5.3	Observation Model	19
5.4	The Predict-Update Cycle	20
5.5	Innovation-Based Gating	20
5.6	No-Arbitrage Constraints	20
5.7	Filtered Estimates Feed the Pricing DAG	21
5.8	Broken States and Cross-Asset Coherence	21
6	The Pricing DAG	22
6.1	Construction Algorithm	22
6.2	DAG Topology by Instrument Class	22
6.3	DAG Mutation	23
7	Temporal Workflow Architecture for Pricing	24
7.1	Design Principle: One Workflow per Unit	24
7.2	Cadence and Heterogeneous Compute	24
7.3	DAG Signalling	24
7.4	Workflow Pseudocode with FSM	24
7.5	Timeout Configuration	25
7.6	Task Queue Architecture	26

7.7	Stale-Price Policy	26
8	PnL Explain as Verification Gate	27
8.1	Mathematical Foundation	27
8.2	The PnL Explain Function	27
8.3	Tolerances	28
8.4	Failure Handling (FSM transitions T6, T7, T11)	28
9	Continuous Approximate Pricing	29
9.1	The Taylor Approximation Formula	29
9.2	Properties	29
9.3	Official PnL Uses FIRM Prices Only	29
10	The Valuation Store	30
11	Edge Cases	31
11.1	Market Data Gap	31
11.2	Model Failure	31
11.3	DAG Mutation Mid-Cycle	31
11.4	Monte Carlo Determinism	31
11.5	Taylor Approximation During Large Moves	31
12	Worked Example: Complete Valuation Cycle	32
12.1	Setup	32
12.2	14:29:45 UTC — Approximate Pricing (FSM: EXPLAINED)	32
12.3	14:30:00 UTC — Full Reprice Cycle	32
12.4	Portfolio Valuation	32
13	Consistency with the Lifecycle Engine	33
13.1	The Price Function Depends on Unit State	33
13.2	The Lifecycle-Valuation Ordering Protocol	33
13.3	Lifecycle Events and the FSM	33
13.4	Taylor Approximation Invalidation	33
13.5	Extended Pricing DAG	33
13.6	Worked Example: Bond Coupon Payment	34
14	Production Scalability Analysis	35
14.1	Workflow Execution Volume	35
14.2	Temporal Cluster Sizing	35
14.3	History Management	35
14.4	Scalability Summary	36
15	Interaction Summary	37
16	Conclusion and Open Problems	38
16.1	Open Problems	38

1 Introduction

The Ledger framework [1] establishes six properties by construction: atomicity, conservation, determinism, state-sufficiency, lifecycle value invariance, and time travel. Portfolio valuation appears in [1, §4] as:

$$V_t = \sum_{u \in \mathcal{U}} w_t(u) \cdot P_t(u)$$

where $w_t(u)$ is the wallet balance (or, in the generalised position model, $\text{own}(u)$) and $P_t(u)$ is the price of unit u at time t in the reference currency. The path-independence theorem [1, §4.3] proves that $\text{PnL} = V_{t_1} - V_{t_0}$ regardless of intermediate trading activity.

The framework is deliberately agnostic to the pricing methodology: “ P_t is an *external input* to the framework, not computed by it” [1, §4.1]. This is the correct architectural boundary—the ledger is a settlement and position system, not a pricing engine. But the boundary creates five unanswered questions:

1. **What does $P_t(u)$ return?** A price is not a scalar in practice. How does structured output interact with the scalar P_t of the PnL theorem?
2. **How are prices computed in dependency order?** Dependencies form a directed acyclic graph. How is it defined, enforced, and executed?
3. **How are new prices validated?** PnL explain provides the mathematical foundation but not the operational pipeline.
4. **What price is available between full reprices?** Complex instruments take seconds to minutes. Between cycles, the risk pipeline needs a current price.
5. **What is the lifecycle of a valuation?** From the arrival of fresh market data through pricing, verification, and publication—what states does a unit’s valuation pass through, and what governs the transitions?

This document answers all five.

1.1 Relationship to the Ledger Specification

This document is a companion to, not a revision of, the Ledger framework [1]. It introduces no new primitives (no new move types, no new wallet structures, no changes to the conservation law). It specifies the *valuation layer* that produces the P_t function consumed by the ledger’s PnL computation.

1.2 Relationship to the Reference Papers

The PnL explain paper [2] establishes the polynomial decomposition of price changes. The PnL attribution and total gamma paper [3] unifies the polynomial framework with smile-adjusted sensitivities and defines the toxic product boundary. The Calibration Manifesto [5] provides the Bayesian framework for market data calibration. The implied volatility [6] and local volatility [7, 8] papers define the models that produce model-dependent Greeks. The control variate paper [9] specifies regression-based Monte Carlo with surrogate models. The total gamma paper [4] derives the adjusted gamma under spot-dependent volatility.

This document operationalises all of these results within a unified valuation architecture.

1.3 Document Roadmap

Section 2 defines the valuation lifecycle FSM. Section 3 defines the ValuationRecord. Section 4 reframes Greeks through the parameter-martingale lens, introduces the sensitivity Jacobian, develops total gamma, and specifies the consensus protocol. Section 5 formalises the Kalman filter for market data calibration. Section 6 introduces the Pricing DAG. Section 7 specifies the Temporal workflow architecture. Section 8 formalises PnL explain. Section 9 introduces continuous approximate pricing. Section 10 describes the valuation store. Section 11 catalogues edge cases. Section 12 traces a complete valuation cycle. Section 13 formalises lifecycle-valuation consistency. Section 14 provides the production scalability analysis. Section 16 identifies open problems.

2 The Valuation Lifecycle FSM

Every unit in the Pricing DAG has a valuation state that governs what operations are permitted and what downstream consumers can expect. The valuation lifecycle is a finite state machine (FSM) with eight states and twelve transitions.

2.1 States

Definition 2.1 (Valuation States). *The valuation lifecycle of a unit u is described by a state $\sigma(u) \in \mathcal{S}$, where:*

$$\mathcal{S} = \{\text{UNPRICED}, \text{PRICING}, \text{PRICED}, \text{EXPLAINING}, \text{EXPLAINED}, \text{QUARANTINED}, \text{STALE}, \text{FAILED}\}$$

Each state has the following semantics:

<i>State</i>	<i>Semantics</i>
UNPRICED	Unit registered in the Unit Store but never priced. No <i>ValuationRecord</i> exists. No approximate price is available. The unit is invisible to the PnL pipeline. This is the initial state for every newly registered unit.
PRICING	A pricing computation is in progress. The pricer activity has been dispatched but has not yet returned. The previous <i>FIRM</i> price (if any) remains current. Approximate pricing from the previous cycle's Greeks is available.
PRICED	The pricer has returned a candidate <i>ValuationRecord</i> . The record has not yet been verified by PnL <i>explain</i> . This is a transient state—the system immediately initiates the <i>explain</i> check.
EXPLAINING	PnL <i>explain</i> is executing. The candidate record is being compared against the previous <i>FIRM</i> record using the Taylor decomposition.
EXPLAINED	PnL <i>explain</i> passed. The <i>ValuationRecord</i> is published to the valuation store with quality = <i>FIRM</i> . Greeks are cached for approximate pricing. This is the normal resting state between pricing cycles.
QUARANTINED	PnL <i>explain</i> failed: the unexplained residual exceeds tolerance. The candidate record is held in quarantine. The previous <i>FIRM</i> price remains current. Automated retry is initiated.
STALE	The staleness timer has expired without a new pricing cycle completing. The last <i>FIRM</i> price is retained with quality = <i>STALE</i> . Downstream consumers apply prudential haircuts.
FAILED	The pricer returned an error, timed out, or produced a numerically invalid result (NaN, Inf, negative price for a non-negative instrument). Retry is initiated with backoff.

Terminal states. There are no terminal states in the FSM during the unit's active lifecycle. Every non-EXPLAINED state has at least one outgoing transition. When the unit's lifecycle terminates (option expiry, bond maturity), the unit exits the Pricing DAG entirely; the FSM ceases to apply, and the terminal valuation is deterministic [1, §7].

2.2 Transitions

Definition 2.2 (Valuation Transitions). *The transition function $\delta : \mathcal{S} \times \mathcal{E} \rightarrow \mathcal{S}$ maps (current state, triggering event) to a new state, subject to guard conditions. The transitions are:*

#	From	To	Trigger	Guard
T1	UNPRICED	PRICING	Market data arrives	Market data fresh and all upstream DAG nodes have a FIRM price (or first cycle)
T2	PRICING	PRICED	Pricer returns	Pricer returns valid result
T3	PRICING	FAILED	Pricer error	Pricer error, timeout, or invalid result
T4	PRICED	EXPLAINING	PnL explain invoked	Always (automatic)
T5	EXPLAINING	EXPLAINED	Explain passes	$ \text{unexplained} < \text{tolerance}(u)$
T6	EXPLAINING	QUARANTINED	Explain fails	$ \text{unexplained} \geq \text{tolerance}(u)$
T7	QUARANTINED	PRICING	Retry initiated	Retry count < max retries and (refreshed data available or alternative model available)
T8	EXPLAINED	STALE	Staleness timer	No new cycle completed within staleness threshold
T9	STALE	PRICING	New market data	Fresh market data arrives
T10	FAILED	PRICING	Retry after backoff	AND upstream nodes fresh Backoff period elapsed AND retry count < max attempts
T11	QUARANTINED	STALE	Retries exhausted	Retry count = max retries
T12	FAILED	STALE	Retries exhausted	and no manual override Retry count = max attempts and no fallback model

Note on T1 guard. For the very first pricing cycle of a unit (state = UNPRICED), the guard on upstream nodes being EXPLAINED is relaxed: it suffices that upstream nodes have *any* FIRM price available, even from a previous cycle. This avoids a bootstrap deadlock where no unit can be priced because none is yet EXPLAINED.

2.3 FSM Diagram

2.4 FSM and the Pricing Workflow

The FSM is implemented within the `PricingWorkflow` (Section 7). The workflow maintains the current state $\sigma(u)$ as a workflow-local variable. Each transition is atomic: the workflow advances the state, performs the associated action, and persists the state via Temporal’s durable execution. On workflow replay, the state is recovered from the event history.

The FSM integrates with the lifecycle engine (Section 13) as follows: lifecycle events that are *material* (Definition 13.3) force the unit into PRICING regardless of current state, because the cached Greeks are invalid after the state change. Non-material lifecycle events do not affect the FSM.

2.5 FSM and the Taylor Approximation

The Taylor approximation (Section 9) is available *only* in state EXPLAINED. In this state, the last FIRM ValuationRecord provides the base price and Greeks, and current market data moves provide the perturbation. The approximate price is:

$$P_{\text{approx}}(u, t) = P_{\text{FIRM}}(u) + \langle \Delta \mid \Delta x(t) \rangle + \frac{1}{2} \langle \Delta x(t) \mid \Gamma \mid \Delta x(t) \rangle$$

where $\Delta x(t)$ is the market data move since the last FIRM snapshot. When the unit transitions out of EXPLAINED (via T8 to STALE, or via a lifecycle event to PRICING), the Taylor approximation is invalidated.

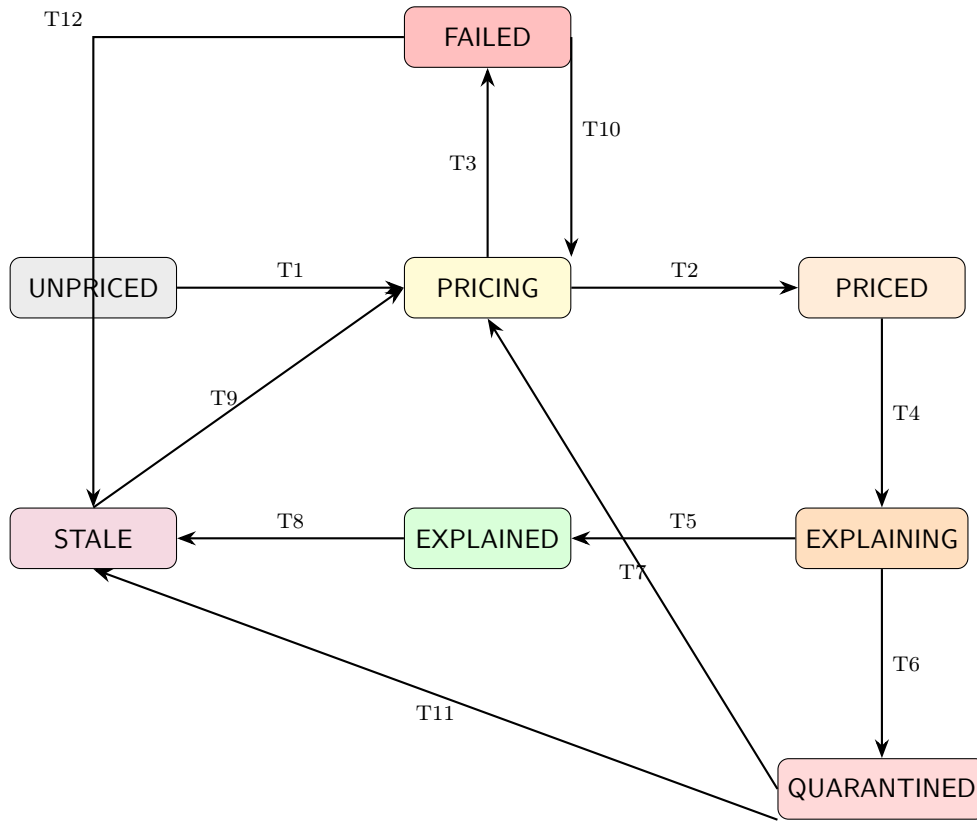


Figure 1: Valuation lifecycle FSM. Green: normal resting state. Yellow/orange: transient processing states. Red: failure states. Purple: degraded state. Gray: initial state.

The FSM thus makes explicit what v0.2 described procedurally: the Taylor approximation is the price in state EXPLAINED between repricing cycles.

3 The Valuation Record

The Ledger specification defines $P_t : \mathcal{U} \rightarrow \mathbb{R}$ as a scalar function. In practice, the output of a pricing computation is richer.

Definition 3.1 (Valuation Record). *A ValuationRecord for unit u at time t is a tuple:*

```
ValuationRecord:
  unit_id:      UnitId
  timestamp:    Timestamp
  dirty_price:  Decimal      -- full market value including accrual
  clean_price:  Decimal      -- quoted price net of accrual
  accrued:     Decimal      -- dirty_price - clean_price
  greeks:      Greeks       -- model-specific sensitivity vector
  model_id:    String       -- which pricer produced this (load-bearing)
  market_data_snap: SnapshotId -- which market data was used
  compute_ms:  Int         -- how long pricing took
  quality:     Quality      -- FIRM | INDICATIVE | APPROXIMATE
                                -- | STALE | FAILED
  fsm_state:   ValuationState -- current FSM state at publication
```

Principle 3.2 (Scalar Compatibility). *The dirty_price field IS the $P_t(u)$ of [1, §4.1]. The path-independence theorem is preserved:*

$$\text{PnL} = V_{t_1} - V_{t_0} = \sum_u \text{own}(u) \cdot [\text{dirty_price}_{t_1}(u) - \text{dirty_price}_{t_0}(u)]$$

The remaining fields are metadata that enriches the record without altering the theorem.

3.1 Quality Semantics

The quality flag has five values with precise operational semantics:

Quality	Semantics
FIRM	Full reprice from live market data; PnL explain passed. FSM state: EXPLAINED.
INDICATIVE	Full reprice completed but PnL explain tolerance exceeded, or market data from secondary source.
APPROXIMATE	Taylor expansion from the last FIRM valuation. FSM state: EXPLAINED (between cycles).
STALE	Last FIRM valuation older than staleness threshold. FSM state: STALE.
FAILED	Pricer failed. FSM state: FAILED.

3.2 Dirty Price, Clean Price, and Accrued Interest

For instruments with accrual mechanics (bonds, loans, swaps):

$$\text{dirty_price} = \text{clean_price} + \text{accrued}$$

The accrued interest is computed from the unit state [1, §7]. For instruments without accrual, $\text{clean_price} = \text{dirty_price}$ and $\text{accrued} = 0$.

4 Greeks, Parameters, and the Sensitivity Jacobian

4.1 Models, Parameters, and the Martingale Assumption

A pricing model $\mathcal{M}$ is defined by three objects:

Definition 4.1 (Pricing Model). *A pricing model $\mathcal{M}$ consists of:*

1. *A set of observables $O = \{S, r, \dots\}$ —quantities directly measured in the market (spot price, deposit rate, exchange rate). These move, and we see them move.*
2. *A set of parameters $\Theta = \{\theta_1, \dots, \theta_n\}$ —quantities calibrated from market data (implied vol, mean reversion speed, vol-of-vol, correlation, curvature coefficients). These are inferred, not directly observed.*
3. *A price function $P_{\mathcal{M}}(O, \Theta, T)$ that maps observables, parameters, and time to a price.*

The distinction between observables and parameters is the distinction between what the model *takes as given* and what the model *infers*. Spot is an observable: it trades on an exchange and has a last price. Implied volatility is a parameter: it is backed out from option prices via an inversion of the Black–Scholes formula. The Kalman filter in Section 5 is the mechanism that performs this inference—it transforms raw observations y_t into calibrated parameter estimates $x_{t|t}$.

Principle 4.2 (The Martingale Assumption on Parameters). *The model assumes that its parameters are martingales:*

$$\mathbb{E}[\Theta(t + dt) \mid \mathcal{F}_t] = \Theta(t) \quad (1)$$

That is: the best forecast of tomorrow’s parameter value is today’s parameter value. This is not the claim that parameters are constant. Parameters can and do change. But the model prices as if the current parameter values are the best estimate of future parameter values.

This is the same assumption that underpins the driftless transition model in the Kalman filter (Eq. 12): $x_t = x_{t-1} + w_t$ with $\mathbb{E}[x_t \mid \mathcal{I}_{t-1}] = x_{t-1}$. It is also Axiom A5 of the Calibration Manifesto [5]: the posterior mean is a martingale. The assumption appears in three places—the transition model, the pricing model, and the Greek computation—because it *is* the same assumption.

4.2 Greeks as Parameter-Invariant Sensitivities

A Greek is a partial derivative of the price function with respect to an observable, holding all parameters at their current values.

Definition 4.3 (Greek). *Let $\mathcal{M}$ be a pricing model with price function $P_{\mathcal{M}}(O, \Theta, T)$. A Greek with respect to observable O_i is:*

$$\left. \frac{\partial P_{\mathcal{M}}}{\partial O_i} \right|_{\Theta}$$

The subscript $|_{\Theta}$ is essential: it asserts that all model parameters are held at their current values. This is well-defined because parameters are martingales—bumping an observable does not, in the model’s worldview, change the parameters.

Delta, gamma, rho, and theta are Greeks in this sense: they are partial derivatives with respect to observables (S, r, t) , holding all model parameters constant. These are well-defined for every model.

But “vega”—the sensitivity to volatility—requires a more careful treatment.

4.3 The Vanishing Vega Problem

In Black–Scholes, volatility σ is a single parameter. The quantity $\partial P/\partial\sigma$ is well-defined: there is one number to bump, and the derivative is a scalar. This is what traders call “vega.”

But in any model where volatility is governed by multiple parameters, the scalar “vega” does not exist. It is replaced by a *vector* of parameter sensitivities whose dimension depends on the model:

Model	Vol parameters	Dim	Vol sensitivities
Black–Scholes	σ_{imp}	1	$\frac{\partial P}{\partial\sigma_{\text{imp}}}$ (scalar “vega”)
Heston	$v_0, \kappa, \theta, \xi, \rho$	5	$\frac{\partial P}{\partial v_0}, \frac{\partial P}{\partial \kappa}, \frac{\partial P}{\partial \theta}, \frac{\partial P}{\partial \xi}, \frac{\partial P}{\partial \rho}$
SABR	α, β, ρ, ν	4	$\frac{\partial P}{\partial \alpha}, \frac{\partial P}{\partial \beta}, \frac{\partial P}{\partial \rho}, \frac{\partial P}{\partial \nu}$
Local vol	$\sigma_{\text{loc}}(K_i, T_j)$ grid	$N_K \times N_T$	$\frac{\partial P}{\partial \sigma_{\text{loc}}(K_i, T_j)}$ (a matrix)
Kernel vol [6]	$\sigma_0, s_0, c_1, \dots, c_n$	$n + 2$	$\frac{\partial P}{\partial \sigma_0}, \frac{\partial P}{\partial s_0}, \frac{\partial P}{\partial c_1}, \dots$

Principle 4.4 (Vega is Model-Specific). *“Vega” is not a universal Greek. It is a model-specific sensitivity whose dimension equals the number of volatility-related parameters in the model. In Black–Scholes, vega is a scalar. In Heston, it is a 5-vector. In local vol, it is a matrix. The scalar “vega” that Bloomberg reports is the Black–Scholes vega— $\partial P/\partial\sigma_{\text{imp}}$ —regardless of which model priced the option. This is a convention, not a fundamental quantity.*

The practical consequences are immediate. If a risk system reports a single “vega” number for a position priced with Heston, it has projected a 5-dimensional sensitivity onto a single axis. The projection discards information about how the position responds to changes in mean reversion, vol-of-vol, or correlation. The discarded information becomes unexplained PnL.

4.4 The Sensitivity Jacobian

The general framework replaces the scalar “vega” with the full parameter Jacobian.

Definition 4.5 (Sensitivity Jacobian). *For a model $\mathcal{M}$ with parameters $\Theta = (\theta_1, \dots, \theta_n)$, the sensitivity Jacobian is the row vector:*

$$J = \left(\frac{\partial P}{\partial \theta_1}, \frac{\partial P}{\partial \theta_2}, \dots, \frac{\partial P}{\partial \theta_n} \right) \in \mathbb{R}^{1 \times n} \quad (2)$$

This is the gradient of price with respect to all model parameters, holding observables constant.

The PnL decomposition becomes:

$$\Delta P \approx \underbrace{\delta \cdot \Delta S}_{\text{delta PnL}} + \underbrace{J \cdot \Delta \Theta}_{\text{parameter PnL}} + \underbrace{\frac{1}{2} \Gamma \cdot (\Delta S)^2}_{\text{gamma PnL}} + \underbrace{\Theta_{\text{decay}} \cdot \Delta t}_{\text{theta PnL}} + \varepsilon \quad (3)$$

where $\Delta \Theta = \Theta_t - \Theta_{t-1}$ is the observed change in calibrated parameters between two valuation times, and $J \cdot \Delta \Theta = \sum_{i=1}^n \frac{\partial P}{\partial \theta_i} \Delta \theta_i$ is the *parameter PnL*.

The “unexplained” residual ε now captures only higher-order terms, cross-gamma terms, and model error. The parameter PnL—which was previously *inside* the unexplained term when using scalar vega—is now explicitly attributed to each parameter that moved.

Remark 4.6 (The scalar vega as a projection). *The scalar “vega” reported by Bloomberg is the inner product $J \cdot \hat{e}_\sigma$, where $\hat{e}_\sigma$ is the unit vector in the direction of a parallel shift of the entire implied vol surface. It is a projection of the full Jacobian $J \in \mathbb{R}^n$ onto a specific direction in parameter space. This projection is useful as a summary statistic but discards all information about non-parallel moves (skew steepening, curvature changes, term structure rolling).*

4.5 Why the Invariant Matters for PnL

Different models have different parameter sets Θ , therefore different Jacobians J , and therefore different PnL decompositions.

Black–Scholes. $\Theta = \{\sigma_{\text{imp}}\}$, so $J = (\nu)$ is the scalar vega. The model holds σ_{imp} constant. But implied volatility is not constant—it has skew and term structure, and the skew moves with spot. The unexplained term includes the skew moving with spot (the “skew delta”), the convexity changes in the smile, and any term structure rolling.

Local volatility. $\Theta = \{\sigma_{\text{loc}}(K_i, T_j)\}$ is the entire local vol grid. When delta is computed by bumping spot, the local vol at the new spot is different (because the local vol surface, derived from the implied vol surface via Dupire’s formula [8, 7], encodes the smile). Local vol delta already accounts for how the vol surface shifts with spot under sticky-local-vol dynamics—the model’s invariant is closer to empirical behaviour than Black–Scholes, so the unexplained PnL is typically smaller.

Heston. $\Theta = \{v_0, \kappa, \theta, \xi, \rho\}$. The Jacobian has five components. The parameter closest to “vega” is $\partial P / \partial v_0$ (sensitivity to initial variance). But $\partial P / \partial \theta$ (long-run variance), $\partial P / \partial \xi$ (vol-of-vol), and $\partial P / \partial \rho$ (spot-vol correlation) also affect the option price through distinct channels. Reporting only $\partial P / \partial v_0$ as “vega” discards the sensitivity to correlation and vol-of-vol changes.

The fundamental insight. The model with the richest parameter set that still matches market behaviour produces the most informative Jacobian and the smallest unexplained PnL. Under-parameterised models (Black–Scholes) lump multiple distinct risks into the unexplained residual. Over-parameterised models (too many local vol nodes) create numerical instability without reducing the residual. The Jacobian dimension is a design choice with direct PnL consequences.

4.6 Black–Scholes Delta vs. Local Vol Delta: A Concrete Comparison

Consider an at-the-money NVDA call with $S = K = 900$, $T = 0.25$, $r = 5\%$, and a market implied volatility of 42%. The equity skew is typical: deep OTM puts trade at higher implied vol than ATM.

Black–Scholes ($\Theta = \{\sigma_{\text{imp}}\}$, held constant). The model assumes a flat, constant volatility surface. The delta is $\delta_{\text{BS}} = N(d_1)$ where d_1 incorporates the single ATM implied vol. For our parameters:

$$\delta_{\text{BS}} = N\left(\frac{\ln(900/900) + (0.05 + 0.42^2/2) \times 0.25}{0.42\sqrt{0.25}}\right) = N(0.164) \approx 0.565$$

This is the sensitivity of price to spot *if you believe the vol parameters do not move when spot moves*. Nobody believes this.

Local volatility ($\Theta = \{\sigma_{\text{loc}}(K, T)\}$ surface, held constant). The local vol surface $\sigma_{\text{loc}}(S, t)$ is derived from the full implied volatility smile via the Dupire formula [8, 7]:

$$\sigma_{\text{loc}}^2(m, T) = \sigma^2 \frac{1 + 2\frac{T}{\sigma} \frac{\partial \sigma}{\partial T}}{1 - 2\frac{m}{\sigma} \frac{\partial \sigma}{\partial m} + \left(\frac{m^2}{\sigma^2} - \frac{\sigma^2 T^2}{4}\right) \left(\frac{\partial \sigma}{\partial m}\right)^2 + \sigma T \frac{\partial^2 \sigma}{\partial m^2}}$$

When delta is computed by bumping the spot in a local vol model, the vol surface *moves with the spot*: as S increases, the option becomes more in-the-money, and the local vol at the new

spot is lower (because of the negative equity skew). This partial offset reduces the effective delta:

$$\delta_{LV} \approx 0.541$$

The difference matters. The delta difference $\delta_{BS} - \delta_{LV} \approx 0.024$ (2.4 delta points) is material for hedging. For a position of 1,000 contracts at \$100 multiplier, the hedge difference is $0.024 \times 1,000 \times 100 \times 900 = \$2,160,000$ in notional—a significant mishedge.

The root cause is the parameter difference: Black–Scholes has one vol parameter; local vol parameterises the entire surface. When spot moves, local vol holds its richer parameter set constant, which implicitly adjusts implied vol according to the smile. If the market’s empirical dynamics are closer to sticky local vol (which they typically are for equities), then δ_{LV} is the better hedge. The `model_id` field on the ValuationRecord tells risk managers which delta—and which parameter set—they are looking at.

4.7 Total Delta: Incorporating Empirical Parameter-Spot Dependence

The martingale assumption says that parameters do not move *in expectation*. But empirically, when spot moves, the vol surface also moves—there is a statistical dependence $\partial\theta_i/\partial S$ that is *not* part of any model’s dynamics but is observed in the data.

The *total delta* captures both the model’s partial delta and this empirical correction:

$$\delta_{\text{total}} = \underbrace{\frac{\partial P}{\partial S} \Big|_{\Theta}}_{\text{partial delta}} + \underbrace{\sum_{i=1}^n \frac{\partial P}{\partial \theta_i} \cdot \frac{\partial \theta_i}{\partial S}}_{\text{parameter response to spot}} \quad (4)$$

The first term is the partial delta (the model’s invariant assumption: parameters do not move). The second term is the “skew delta”—how the price changes because parameters *empirically* respond to spot moves. The terms $\partial\theta_i/\partial S$ are *not* model-derived; they come from observing how calibrated parameters change when spot changes.

In the simplest case—Black–Scholes with one vol parameter—this reduces to [3, 4]:

$$\frac{dP}{dS} = \Delta + \text{Vega} \cdot \frac{d\sigma}{dS} \quad (5)$$

where the “skew delta” term $\text{Vega} \cdot d\sigma/dS$ vanishes only if σ is truly independent of S .

A model that already captures the spot-vol correlation in its dynamics (like local vol or Heston with $\rho \neq 0$) will have a partial delta closer to the total delta. Black–Scholes, which assumes zero spot-vol correlation, will have a large skew correction.

4.8 Total Gamma Under Smile-Dependent Volatility

When implied volatility depends on spot price through the smile, the classical partial-derivative gamma (computed holding σ constant) understates the true convexity. The total gamma incorporates the indirect effect of spot on price through volatility [3, 4]:

$$\Gamma_{\text{total}} = \Gamma + 2 \text{Vanna} \cdot \frac{d\sigma}{dS} + \text{Volga} \cdot \left(\frac{d\sigma}{dS} \right)^2 + \text{Vega} \cdot \frac{d^2\sigma}{dS^2} \quad (6)$$

Each term has a transparent origin:

Γ	direct spot convexity (partial gamma, parameters held constant)
$2 \text{Vanna} \cdot \sigma'$	cross-coupling of spot and vol (two chain-rule pathways)
$\text{Volga} \cdot (\sigma')^2$	vol convexity channelled through smile slope
$\text{Vega} \cdot \sigma''$	price-vol sensitivity weighted by smile curvature

This is the total second derivative of price with respect to spot, including the effect of vol parameters moving with spot. Using Γ_{total} instead of Γ_{partial} in the second-order PnL term captures the vol-spot correlation and shrinks the unexplained residual.

4.8.1 Volatility Decomposition: ATM and Smile

In practice [3, 4], $\sigma(S)$ is modelled as:

$$\sigma(S) = \sigma_{\text{ATM}}(m) + \sigma_{\text{smile}}(k) \quad (7)$$

where $m = \ln(S/S_0)$ is the log-moneyness of spot relative to a pivot and $k = \ln(K/S)$ is the log-moneyness of the strike relative to spot. The key observation is that m and k respond oppositely to spot moves: $dm/dS = +1/S$, $dk/dS = -1/S$.

The total first derivative of implied vol with respect to spot is:

$$\frac{d\sigma}{dS} = \frac{\sigma'_{\text{ATM}}(m) - \sigma'_{\text{smile}}(k)}{S} \quad (8)$$

and the vega-curvature contribution is:

$$\text{Vega} \cdot \frac{d^2\sigma}{dS^2} = \frac{\text{Vega}}{S^2} [\sigma''_{\text{ATM}}(m) - \sigma'_{\text{ATM}}(m) + \sigma''_{\text{smile}}(k) + \sigma'_{\text{smile}}(k)] \quad (9)$$

Every quantity in this expression is observable or computable from the volatility surface model [6, 7].

4.9 PnL Attribution with the Jacobian

The polynomial PnL framework [2, 3] provides the *form* of the decomposition; the Jacobian and total Greeks provide the *content*. The full PnL attribution is:

$$\Delta P = \underbrace{\delta \cdot \Delta S}_{\text{delta}} + \underbrace{J \cdot \Delta \Theta}_{\text{parameter}} + \underbrace{\frac{1}{2} \Gamma \cdot (\Delta S)^2}_{\text{gamma}} + \underbrace{\Theta_{\text{decay}} \cdot \Delta t}_{\text{theta}} + \underbrace{\varepsilon}_{\text{unexplained}} \quad (10)$$

The parameter PnL $J \cdot \Delta \Theta$ is the sum over all model parameters of their sensitivity times their observed change. For a Black–Scholes model, this is $\nu \cdot \Delta \sigma_{\text{imp}}$. For Heston, it is $\frac{\partial P}{\partial v_0} \Delta v_0 + \frac{\partial P}{\partial \kappa} \Delta \kappa + \frac{\partial P}{\partial \theta} \Delta \theta + \frac{\partial P}{\partial \xi} \Delta \xi + \frac{\partial P}{\partial \rho} \Delta \rho$. For the kernel vol model [6], it is $\frac{\partial P}{\partial \sigma_0} \Delta \sigma_0 + \frac{\partial P}{\partial s_0} \Delta s_0 + \sum_i \frac{\partial P}{\partial c_i} \Delta c_i$.

The unexplained residual ε now captures only:

- Higher-order Taylor terms (cubic and beyond)
- Cross-gamma terms ($\partial^2 P / \partial S \partial \theta_i$, i.e., vanna, charm)
- Model error (the model's price function does not perfectly reproduce the market)

When $\sigma = \sigma(S)$, the gradient and Hessian in the polynomial PnL identity must be the total derivatives:

$$P(S_2) - P(S_1) = \frac{1}{2} \left(\left. \frac{dP}{dS} \right|_{S_2} + \left. \frac{dP}{dS} \right|_{S_1} \right) h - \frac{1}{12} \left(\left. \frac{d^2 P}{dS^2} \right|_{S_2} - \left. \frac{d^2 P}{dS^2} \right|_{S_1} \right) h^2 \quad (11)$$

Using partial Greeks in this formula—omitting the smile adjustments—produces a spurious unexplained residual equal to the PnL arising from the violated martingale assumption on parameters.

Remark 4.7 (Unexplained PnL as parameter martingale violation). *Let δ_{partial} denote the partial delta (parameters held constant) and δ_{total} the total delta accounting for the empirical $\partial \Theta / \partial S$. Over a spot move ΔS , the hedge error from using δ_{partial} is:*

$$(\delta_{\text{total}} - \delta_{\text{partial}}) \cdot \Delta S = \sum_i \frac{\partial P}{\partial \theta_i} \cdot \frac{\partial \theta_i}{\partial S} \cdot \Delta S$$

This is exactly the PnL arising from the model’s martingale assumption ($\partial\theta_i/\partial S = 0$) being violated (θ_i moved with spot). The Jacobian framing makes the source of every component of unexplained PnL transparent.

4.10 The Toxic Product Boundary

The cubic polynomial assumption is a deliberate risk management boundary [3].

Definition 4.8 (Toxic product). *A derivative product is toxic if the residual $|R(x_1, x_2)| = |[P(x_2) - P(x_1)] - [P_3(x_2) - P_3(x_1)]| \geq \varepsilon$ for a tolerance ε over plausible market moves, where P_3 is the degree-three Taylor polynomial.*

Within the non-toxic boundary, the third derivative $\Lambda = P'''$ is constant, Γ is linear, and Δ is quadratic. Every PnL identity is *exact*—not approximate. Products breaching the boundary (e.g., digital options near expiry, barrier options near trigger) exhibit sensitivity structures that cannot be hedged at standard rebalancing frequencies. The cubic residual serves as a diagnostic: a large unexplained PnL signals that the product lies outside the boundary of manageable risk.

4.11 The Consensus Problem: Multiple Models

When multiple models are run on the same instrument, they produce different Jacobians because their parameter sets differ. This creates four operational questions, each with a distinct answer:

1. **Which delta do you hedge with?** Use the primary model—the one whose parameter set best matches empirical dynamics. For equities with material skew, this is typically calibrated local vol or a calibrated stochastic vol model. The primary model is designated per instrument class and calibrated to the live vol surface.
2. **Which Greeks do you use for PnL explain?** The *same* model that produced the price (model-consistent Jacobian). Mixing models—using model $\mathcal{M}_1$ ’s Jacobian to explain model $\mathcal{M}_2$ ’s price change—creates a spurious unexplained residual that is a model artefact, not a market phenomenon (Remark 8.1).
3. **Which price do you report?** The primary model’s price. The model risk reserve equals the maximum price difference across approved models:

$$\text{ModelReserve}(u, t) = \max_{i,j} |P_{\mathcal{M}_i}(u, t) - P_{\mathcal{M}_j}(u, t)|$$

4. **Which Greeks do you use for risk limits?** The most conservative—largest absolute delta for delta limits, largest VaR for capital. This ensures that limits are not breached by model choice.

Firms typically maintain:

- A *primary model* (e.g., calibrated local vol) for hedging and official PnL.
- A *secondary model* (e.g., SABR) for model risk assessment.
- A *stress model* (e.g., Black–Scholes with shocked vol) for regulatory capital.

The valuation store holds one ValuationRecord per (unit, timestamp, model), so all sets of Greeks coexist.

4.12 Practical Computation of Greeks

Greeks—both observable sensitivities and the parameter Jacobian—are computed by one of four methods, each with different trade-offs:

Analytical. Available for Black–Scholes and a few other closed-form models. Fast (<1ms) but limited to models with closed-form price functions. Delta, gamma, vega, theta, rho all have closed-form expressions in Black–Scholes.

Bump-and-revalue. General-purpose. Bump each input by ε , reprice, compute the central difference:

$$\frac{\partial P}{\partial \theta_i} \approx \frac{P(\theta_i + \varepsilon) - P(\theta_i - \varepsilon)}{2\varepsilon}$$

For the full Jacobian with n parameters plus k observables: $2(n + k)$ reprices. For a Heston model ($n = 5$, $k = 2$): 14 reprices. For a Monte Carlo pricer at 10 seconds per call, full Jacobian computation takes 2–4 minutes.

Adjoint algorithmic differentiation (AAD). Computes the *entire* Jacobian—all observable sensitivities and all parameter sensitivities—in one backward pass through the pricing algorithm. Cost: approximately 3–4× the base price computation, *regardless of the number of parameters*. This makes AAD particularly valuable for models with many parameters (local vol grid, kernel vol coefficients).

Pathwise and likelihood ratio methods (Monte Carlo). Alternatives to bump-and-revalue for MC pricers. The pathwise method differentiates the payoff along each simulated path; the likelihood ratio method differentiates the probability measure rather than the payoff. Each has bias/variance trade-offs: pathwise is unbiased for smooth payoffs but fails at discontinuities (digitals); likelihood ratio handles discontinuities but has higher variance. Greek and Jacobian computation is a separate Temporal activity from the base price (Section 7), with its own timeout and retry policy.

4.13 Greeks Structures by Instrument Class

The `greeks` field of the `ValuationRecord` is a tagged union whose structure is model-dependent. This is a direct consequence of Principle 4.4: the dimension of the sensitivity vector equals $|\text{observables}| + |\text{parameters}|$, which varies by model.

```
Greeks = BSOptionGreeks | HestonOptionGreeks | LocalVolGreeks
        | SABROptionGreeks | KernelVolGreeks
        | IRSGreeks | BondGreeks | FuturesGreeks | CashGreeks
```

4.13.1 Black–Scholes Option Greeks ($\dim \Theta = 1$)

```
BSOptionGreeks:
  model_id:      "black_scholes"
  invariant:     "sigma_imp"
  method:       Method    -- ANALYTICAL | BUMP | AAD | PATHWISE
  bump_size:    Decimal?  -- epsilon used (for BUMP method)
  -- Observable sensitivities
  delta:        Decimal   -- dP/dS, holding sigma_imp constant
  gamma:        Decimal   -- d^2P/dS^2
  theta:        Decimal   -- dP/dt (per calendar day)
  rho:          Decimal   -- dP/dr
  -- Parameter Jacobian (dim = 1)
  vega:         Decimal   -- dP/d(sigma_imp): the scalar "vega"
  -- Cross-sensitivities (for total gamma computation)
  vanna:        Decimal   -- d^2P/(dS d(sigma_imp))
  volga:        Decimal   -- d^2P/d(sigma_imp)^2
```


4.13.2 Heston Option Greeks ($\dim \Theta = 5$)

```
HestonOptionGreeks:
  model_id:      "heston"
  invariant:     "heston_params"
  method:        Method      bump_size:  Decimal?
  -- Observable sensitivities
  delta:         Decimal      gamma:      Decimal
  theta:         Decimal      rho_rate:   Decimal  -- dP/dr
  -- Parameter Jacobian (dim = 5)
  v0_sens:       Decimal      -- dP/d(v0): closest analogue to BS vega
  kappa_sens:    Decimal      -- dP/d(kappa): mean reversion sensitivity
  theta_long_sens: Decimal    -- dP/d(theta): long-run variance sens.
  xi_sens:       Decimal      -- dP/d(xi): vol-of-vol sensitivity
  rho_sens:      Decimal      -- dP/d(rho): spot-vol correlation sens.
```

4.13.3 SABR Option Greeks ($\dim \Theta = 4$)

```
SABROptionGreeks:
  model_id:      "sabr"
  invariant:     "sabr_params"
  method:        Method      bump_size:  Decimal?
  -- Observable sensitivities
  delta:         Decimal      gamma:      Decimal
  theta:         Decimal      rho_rate:   Decimal
  -- Parameter Jacobian (dim = 4)
  alpha_sens:    Decimal      -- dP/d(alpha): closest to BS vega
  beta_sens:     Decimal      -- dP/d(beta): backbone sensitivity
  rho_sens:      Decimal      -- dP/d(rho): correlation sensitivity
  nu_sens:       Decimal      -- dP/d(nu): vol-of-vol sensitivity
```

4.13.4 Local Vol / Kernel Vol Greeks ($\dim \Theta = N_K \times N_T$ or $n + 2$)

```
LocalVolGreeks:
  model_id:      "local_vol" | "kernel_vol"
  invariant:     "sigma_loc_surface" | "kernel_params"
  method:        Method      bump_size:  Decimal?
  -- Observable sensitivities
  delta:         Decimal      gamma:      Decimal
  theta:         Decimal      rho_rate:   Decimal
  -- Parameter Jacobian (dim = grid size or kernel count + 2)
  bucket_vegas:  Map<(Strike, Expiry), Decimal>
                -- dP/d(sigma_loc(K_i,T_j)) for each grid point
                -- OR dP/d(c_i) for each kernel coefficient
```

The bucket vegas form a matrix (or tensor) of sensitivities. For a typical equity surface with 10 strikes and 8 expiries, this is an 80-element Jacobian per instrument. The “parallel vega” (sensitivity to shifting the entire surface by 1 vol point) is the sum of all bucket vegas.

4.13.5 IRS Greeks

```
IRSGreeks:
  model_id:      String
  dv01:          Decimal
  convexity:     Decimal
  key_rate_durations: Map<Tenor, Decimal>
```

The key rate durations are the interest rate Jacobian: $\partial P / \partial r_\tau$ for each tenor τ . PnL explain: explained $\approx \sum_\tau \text{KRD}_\tau \cdot \Delta r_\tau + \frac{1}{2} \cdot \text{convexity} \cdot (\Delta r_{\text{par}})^2$.

4.13.6 Bond Greeks

```
BondGreeks:
  model_id:      String
  modified_duration: Decimal
  convexity:     Decimal
  dv01:         Decimal
```

5 Kalman Filter for Market Data Calibration

The pricing DAG requires calibrated market data: yield curves, volatility surfaces, credit curves, and correlation parameters. Raw market observables—swap rates, cap prices, option prices, bond yields—must be transformed into calibrated parameter vectors that satisfy no-arbitrage constraints. The Calibration Manifesto [5] provides the Bayesian framework; this section specifies the Kalman filter implementation that feeds calibrated data into the valuation engine.

5.1 State Vector

Definition 5.1 (Calibration State). *The calibration state is a latent parameter vector $x_t \in \mathbb{R}^d$ representing the current market state. Examples:*

<i>Calibrated object</i>	<i>State vector x_t</i>	<i>Dimension d</i>
<i>USD yield curve</i>	<i>Zero rates at tenors 1M,3M,6M,1Y,...,30Y</i>	<i>12–15</i>
<i>Equity vol surface (per expiry)</i>	<i>ATM vol, skew, curvature coefficients $(\sigma_0, s_0, c_1, \dots, c_n)$</i>	<i>5–11</i>
<i>Credit hazard curve</i>	<i>Hazard rates at 1Y,3Y,5Y,7Y,10Y</i>	<i>5</i>
<i>FX vol surface</i>	<i>ATM vol, risk reversal, butterfly per expiry</i>	<i>3–5 per expiry</i>

For the implied volatility model [6, 7], the state vector at each expiry is the parameter ket $|\beta\rangle = (\sigma_0, s_0, c_1, \dots, c_n)^\top$, where σ_0 is the ATM vol, s_0 the ATM skew, and c_i the curvature coefficients of the Epanechnikov kernel decomposition. The implied vol is $\sigma(m) = \langle \phi(m) | \beta \rangle$, linear in the parameters—enabling exact Kalman updates without linearisation.

5.2 Transition Dynamics

Consistent with Axiom A5 of [5] (posterior mean is a martingale), the transition model is driftless:

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q) \quad (12)$$

where Q is the process noise covariance. The transition matrix is the identity: $A = I_d$. This ensures:

$$\mathbb{E}[x_t | I_{t-1}] = x_{t-1}$$

Parameters move only because new information arrives. The covariance Q controls how fast the state adapts to new observations: large Q allows rapid adaptation (suitable for volatile markets); small Q produces smoother, more stable curves.

Process noise calibration. Q is diagonal or near-diagonal for yield curves (tenor-by-tenor variance). For vol surfaces, Q may encode correlation between ATM vol and skew parameters. Typical daily process noise standard deviations: 1–3 bp for yield curve nodes, 0.5–1.5 vol points for equity ATM vol, 0.1–0.3 for skew parameters.

5.3 Observation Model

At knowledge time t , a vector of market observables $y_t \in \mathbb{R}^{m_t}$ is observed:

$$y_t = H_t x_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_t) \quad (13)$$

where H_t maps state components to observed instruments and R_t encodes observation noise.

Observation noise. R_t is derived from market microstructure:

- For swap rates: $R_{ii} \approx (\text{half bid-ask spread})^2 \approx (0.25 \text{ bp})^2$.
- For option prices: $R_{ii} \approx (\text{half bid-ask/vega})^2$ (converted to vol units).
- For stale quotes: R_{ii} is inflated by a staleness factor proportional to time since last update.

Linearity. For yield curves (discount factors as state, swap rates as observations), H_t is a linear matrix. For the kernel-based vol model [6], $\sigma(m) = \langle \phi(m) | \beta \rangle$ is linear in β , so H_t is the design matrix Φ evaluated at the observed strikes. For nonlinear observation models (e.g., cap prices as functions of a vol surface), the Extended Kalman Filter (EKF) or Unscented Kalman Filter (UKF) replaces the linear update.

5.4 The Predict-Update Cycle

Definition 5.2 (Kalman Predict-Update). *The Kalman filter proceeds in two steps at each observation epoch:*

Predict. Project the state forward using the transition model:

$$\hat{x}_{t|t-1} = x_{t-1|t-1} \quad (14)$$

$$P_{t|t-1} = P_{t-1|t-1} + Q \quad (15)$$

where $\hat{x}_{t|t-1}$ is the predicted state mean and $P_{t|t-1}$ the predicted state covariance.

Update. Incorporate the new observations:

$$\nu_t = y_t - H_t \hat{x}_{t|t-1} \quad (\text{innovation}) \quad (16)$$

$$S_t = H_t P_{t|t-1} H_t^\top + R_t \quad (\text{innovation covariance}) \quad (17)$$

$$K_t = P_{t|t-1} H_t^\top S_t^{-1} \quad (\text{Kalman gain}) \quad (18)$$

$$x_{t|t} = \hat{x}_{t|t-1} + K_t \nu_t \quad (\text{updated mean}) \quad (19)$$

$$P_{t|t} = (I - K_t H_t) P_{t|t-1} \quad (\text{updated covariance}) \quad (20)$$

The innovation ν_t measures the surprise in the new observation. The Kalman gain K_t allocates trust between the model prediction and the observation, weighted by their respective uncertainties.

5.5 Innovation-Based Gating

Following [5], the squared Mahalanobis distance

$$D_t^2 = \nu_t^\top S_t^{-1} \nu_t$$

is monitored. Under the correct model, $D_t^2 \sim \chi_{m_t}^2$. A threshold u_{m_t} (e.g., 99th percentile) gates observations:

- $D_t^2 \leq u_{m_t}$: observation accepted; standard Kalman update proceeds.
- D_t^2 moderately above u_{m_t} : observation down-weighted by inflating R_t .
- D_t^2 far above u_{m_t} : observation rejected; posterior unchanged: $x_{t|t} = \hat{x}_{t|t-1}$, $P_{t|t} = P_{t|t-1}$. A rejection record is emitted.

This protects the calibration from fat-finger trades, feed outages, and stale quotes mislabelled as current [5, A7].

5.6 No-Arbitrage Constraints

The admissible region Θ_{AF} [5, §5] is enforced as a post-update projection:

$$x_{t|t}^{\text{certified}} = \arg \min_{x \in \Theta_{\text{AF}}} \|x - x_{t|t}\|_{P_{t|t}^{-1}}^2 \quad (21)$$

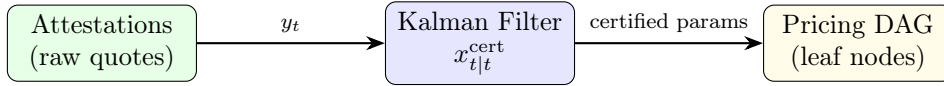
For yield curves: $D(0) = 1$, $D(t) > 0$, $D(t)$ non-increasing (non-negative forward rates). For vol surfaces: $w(x, T) > 0$, no calendar spread arbitrage ($\partial w / \partial T \geq 0$), no butterfly arbitrage (non-negative risk-neutral density). For the kernel-based vol model [7], the butterfly constraint

translates to a positivity condition on the denominator of the Dupire formula, expressible as linear or low-degree polynomial inequalities on the curvature coefficients.

A calibration to x^* is certified only if: (1) $x^* \in \Theta_{\text{AF}}$, (2) per-instrument residuals $|r_i| \leq \tau_i$, and (3) aggregate WRMSE $\leq \tau_{\text{agg}}$ [5, A4]. Failure triggers fallback to the last certified state with staleness flagging.

5.7 Filtered Estimates Feed the Pricing DAG

The output of the Kalman filter—the posterior mean $x_{t|t}^{\text{certified}}$ —is the “best estimate” of the current market state. This feeds into the Pricing DAG (Section 6) as the calibrated market data at the leaf nodes:



The Kalman filter runs as a dedicated Temporal workflow per calibrated object, signalling downstream pricing workflows when a new certified calibration is available—exactly like a market data node in the Pricing DAG. The calibration workflow’s cadence matches the observation frequency: real-time for liquid markets, periodic for illiquid curves.

5.8 Broken States and Cross-Asset Coherence

When multiple calibrated objects share information (e.g., SPX vol and SX5E vol are correlated), asynchronous updates can create broken states [5, §7]: the stored pair $(\sigma_{\text{SPX}}, \sigma_{\text{SX5E}})$ no longer equals the conditional expectation given the full information set. This is operationally important for cross-asset products (spread options, baskets).

The valuation stack handles this by: (1) running joint Kalman filters for correlated parameters where cross-asset products exist, (2) flagging pricing of cross-asset products as INDICATIVE when constituent calibrations are asynchronous beyond a configured threshold, and (3) monitoring the innovation statistics for cross-asset consistency.

6 The Pricing DAG

Pricing is not a flat operation over independent instruments. Dependencies form a directed acyclic graph—the *Pricing DAG*.

Definition 6.1 (Pricing DAG). *The Pricing DAG is a directed acyclic graph $G = (N, E)$ where:*

- $N = N_U \cup N_M \cup N_C$: nodes are unit nodes (N_U), market data nodes (N_M), or calibration nodes (N_C , the Kalman filter outputs from Section 5).
- $E \subseteq N \times N$: edge (a, b) means “ b requires a fresh value from a .”

Calibration nodes sit between raw market data nodes and unit nodes: raw quotes $\rightarrow$ Kalman filter $\rightarrow$ calibrated parameters $\rightarrow$ pricing models $\rightarrow$ unit valuations.

6.1 Construction Algorithm

```
BuildPricingDAG(unit_store: UnitStore) -> DAG:
    dag = empty DAG

    -- Step 1: Add market data nodes (leaf nodes)
    for each observable o referenced by any unit in unit_store:
        dag.add_node(o, type=MARKET_DATA, dependencies=[])

    -- Step 2: Add calibration nodes
    for each calibrated object c (yield curve, vol surface, ...):
        dag.add_node(c, type=CALIBRATION,
                     dependencies=c.input_observables())

    -- Step 3: Add unit nodes with dependencies
    for each unit u in unit_store where u.lifecycle_stage == ACTIVE:
        deps = u.smart_contract.pricing_dependencies()
        dag.add_node(u, type=UNIT, dependencies=deps)

    -- Step 4: Verify acyclicity
    if dag.has_cycle():
        raise DAGCycleError(dag.find_cycle())

    -- Step 5: Compute topological order
    dag.topo_order = topological_sort(dag)
    return dag
```

Invariant 6.2 (Acyclicity). *The Pricing DAG is acyclic. Enforced at unit registration: the Unit Store rejects units whose dependencies would create a cycle.*

6.2 DAG Topology by Instrument Class

Instrument	Upstream dependencies	DAG depth
Cash (USD)	None	0
Equity spot	Exchange price feed	1
Yield curve (calibrated)	Deposit, futures, swap rates	1 (calibration)
Vol surface (calibrated)	Option prices, ATM vol quotes	1 (calibration)
Vanilla option	Spot, calibrated vol surface, calibrated rate curve	2–3
IRS	Calibrated USD yield curve	2
Structured note	Embedded option valuation, credit curve	3+

6.3 DAG Mutation

The Pricing DAG changes when units are added or removed. New units extend the DAG (with acyclicity verification); terminated units are removed. Mid-cycle mutations take effect on the next cycle—a single cycle always operates on a frozen topology.

7 Temporal Workflow Architecture for Pricing

7.1 Design Principle: One Workflow per Unit

Each unit in the Pricing DAG has a dedicated `PricingWorkflow(unit_id)` that:

1. waits for all upstream dependencies to signal fresh prices,
2. retrieves market data and unit state (lifecycle-before-valuation ordering),
3. invokes the pricer activity (base price),
4. dispatches Greek computation activities (bump-and-revalue fan-out),
5. runs PnL explain verification,
6. advances the FSM state (Section 2),
7. publishes the `ValuationRecord`,
8. updates the approximation service with fresh Greeks.

7.2 Cadence and Heterogeneous Compute

Instrument class	Cadence	Compute budget	Pricer type
Equity spot	1–5s	< 1ms	Trivial (pass-through)
Vanilla option	10–30s	1–10ms	Analytical (Black-Scholes)
Exotic option	1–5min	1–30s	Monte Carlo / PDE
IRS	30s–2min	10–100ms	Curve bootstrapping
Bond	30s–2min	1–10ms	Yield-to-maturity
Structured note	5–15min	1–60s	Composite

7.3 DAG Signalling

Reactive pricing via Temporal signals:

1. Leaf node (market data feed or Kalman filter output) signals downstream unit workflows.
2. Unit workflow produces `ValuationRecord`, signals downstream dependents.
3. Downstream workflow checks freshness map: fire pricing only when ALL upstream nodes are fresh.

7.4 Workflow Pseudocode with FSM

```
PricingWorkflow(unit_id, cadence, dependencies):
    fsm_state = UNPRICED
    prev_record = None
    freshness = {dep: None for dep in dependencies}

    while unit_state(unit_id).lifecycle_stage == ACTIVE:
        -- Wait for upstream signals, lifecycle signals, or cadence timer
        await_signals_or_timer(freshness, cadence)

        -- Drain lifecycle signals (lifecycle-before-valuation)
        drain_lifecycle_signals(unit_id)

        if not all_fresh(freshness, cadence):
            continue

        -- T1 or T9 or T10: transition to PRICING
        fsm_state = PRICING

        -- Invoke base pricer
        result = await InvokePricer(unit_id, mkt_snap)
        if result.is_error():
```



```

        fsm_state = FAILED      -- T3
        await backoff_and_retry()
        continue

fsm_state = PRICED            -- T2

-- Compute Greeks
greeks = await ComputeGreeks(unit_id, mkt_snap, result)
raw_record = ValuationRecord(result, greeks, ...)

-- T4: transition to EXPLAINING
fsm_state = EXPLAINING
explain = PnLExplain(prev_record, raw_record, market_moves)

if explain.status == PASS:
    fsm_state = EXPLAINED -- T5
    await PublishValuationRecord(raw_record.with_quality(FIRM))
    prev_record = raw_record
    await UpdateApproxCache(unit_id, raw_record)
    signal_downstream(unit_id, raw_record)
else:
    fsm_state = QUARANTINED -- T6
    await Quarantine(raw_record, explain)
    -- T7: retry with refreshed data
    retry_result = await RetryPricing(unit_id)
    if retry_result.explain_passes:
        fsm_state = EXPLAINED -- T5 via T7
        await PublishValuationRecord(retry_result.record)
    else:
        fsm_state = STALE -- T11
        await AlertRiskTeam(unit_id, explain)

-- Staleness monitoring (T8)
start_staleness_timer(cadence * staleness_factor)

```

7.5 Timeout Configuration

```

PricerActivityOptions(instrument_class):
    match instrument_class:
        case EQUITY_SPOT:
            StartToCloseTimeout: 5s
            ScheduleToCloseTimeout: 30s
        case VANILLA_OPTION:
            StartToCloseTimeout: 30s
            ScheduleToCloseTimeout: 2min
        case EXOTIC_OPTION:
            StartToCloseTimeout: 5min
            ScheduleToCloseTimeout: 15min
        case IRS | BOND:
            StartToCloseTimeout: 2min
            ScheduleToCloseTimeout: 5min
    RetryPolicy:
        InitialInterval: 1s
        BackoffCoefficient: 2.0
        MaximumAttempts: 3
        NonRetryableErrors: [ModelNotFound, UnitNotActive]

```

7.6 Task Queue Architecture

Task queue	Instruments	Worker profile
pricing-fast	Equity spot, FX, cash	Low-latency, high-throughput
pricing-analytical	Vanilla options, bonds, IRS	Medium compute
pricing-mc	Exotic options, structured notes	GPU-enabled, high memory
pricing-greeks	Bump-and-revalue activities	Parallel, medium compute
pricing-calibration	Kalman filter updates	Medium compute, low latency
pricing-workflow	All PricingWorkflow orchestrators	Lightweight, many instances

7.7 Stale-Price Policy

1. **Within cadence window:** workflow waits. FSM stays in EXPLAINED.
2. **Beyond cadence ($\leq 3\times$):** last FIRM price retained, **quality** = STALE. FSM transitions to STALE via T8.
3. **Beyond $3\times$ cadence:** alert raised via obligation-liveness framework [1, §14.7].

8 PnL Explain as Verification Gate

Every new ValuationRecord must pass through PnL explain before publication. This section connects the mathematical foundation of [2, 3] to the FSM transitions T4–T6.

8.1 Mathematical Foundation

For a pricing function P treated as a cubic polynomial in market data vector $x \in \mathbb{R}^n$ [2, 3]:

Ex post (exact for cubic P).

$$P(x_2) - P(x_1) = \left\langle \frac{\Delta_{x_2} + \Delta_{x_1}}{2} \middle| x_2 - x_1 \right\rangle - \frac{1}{12} \langle x_2 - x_1 | (\Gamma_{x_2} - \Gamma_{x_1}) | x_2 - x_1 \rangle \quad (22)$$

Decomposition into three orders.

$$P(x_2) - P(x_1) = \underbrace{\langle \Delta_{x_1} | h \rangle}_{\text{1st order}} + \underbrace{\left\langle \frac{\Delta_{x_2} - \Delta_{x_1}}{2} \middle| h \right\rangle}_{\text{2nd order}} - \underbrace{\frac{1}{12} \langle h | (\Gamma_{x_2} - \Gamma_{x_1}) | h \rangle}_{\text{3rd order}} \quad (23)$$

The Greeks Δ_{x_1} and Δ_{x_2} must come from the same model $\mathcal{M}$. When using instruments with smile-dependent volatility, these must be the *total* Greeks (Section 4.8).

Remark 8.1 (Model Consistency in PnL Explain). *Using model $\mathcal{M}_1$'s Greeks to explain model $\mathcal{M}_2$'s price change produces a residual that is a model artefact. When a model transition occurs, the PnL explain gate is suspended for the transition record, and the model change generates an explicit “model change PnL” reported separately.*

8.2 The PnL Explain Function

Definition 8.2 (PnL Explain). `pnl_explain(`
`prev: ValuationRecord, -- FSM was EXPLAINED`
`curr: ValuationRecord, -- FSM is PRICED`
`market_moves: MarketDataDelta,`
`cashflows: List[Move] -- lifecycle cash flows`
`) -> ExplainResult`

`precondition: prev.model_id == curr.model_id`

The computation:

```
total_pnl = curr.dirty_price - prev.dirty_price
explained = (
    prev.greeks.delta * market_moves.spot_change
    + 0.5 * prev.greeks.gamma * market_moves.spot_change^2
    + prev.greeks.vega * market_moves.vol_change
    + prev.greeks.theta * dt
    + prev.greeks.rho * market_moves.rate_change
    + sum(cf.amount for cf in cashflows))
unexplained = total_pnl - explained
if abs(unexplained) < tolerance(unit_id):
    return ExplainResult(status=PASS, ...) -- enables T5
else:
    return ExplainResult(status=FAIL, ...) -- triggers T6
```

8.3 Tolerances

Instrument class	Tolerance (per unit notional)
Equity spot	\$0.01 (essentially zero)
Vanilla option	1% of option premium
Exotic option	2% of option premium
IRS	0.5 basis points of notional
Bond	0.1% of face value

8.4 Failure Handling (FSM transitions T6, T7, T11)

When PnL explain fails (T6: EXPLAINING $\rightarrow$ QUARANTINED):

1. **Quarantine.** Candidate record held; previous FIRM price remains current.
2. **Alert.** Event-triggered obligation with deadline $3\times$ cadence.
3. **Retry (T7).** Re-invoke with refreshed market data or alternative model.
4. **Manual override.** If automated retries fail, manual price with `quality = INDICATIVE`.
5. **Exhaustion (T11).** Last-known-good with `quality = STALE`.

9 Continuous Approximate Pricing

Principle 9.1 (Price Availability). *At any time t , for any active unit u in FSM state EXPLAINED, a price is always available via Taylor expansion from the last FIRM valuation.*

9.1 The Taylor Approximation Formula

Let t_{last} be the timestamp of the last FIRM valuation (FSM entered EXPLAINED). At time $t > t_{\text{last}}$:

$$P_{\text{approx}}(u, t) = P_{\text{FIRM}}(u) + \delta \cdot \Delta S(t) + \frac{1}{2} \Gamma \cdot \Delta S(t)^2 + \nu \cdot \Delta \sigma(t) + \Theta \cdot \Delta t + \rho \cdot \Delta r(t) \quad (24)$$

For instruments with multiple risk factors:

$$P_{\text{approx}}(u, t) = P_{\text{FIRM}}(u) + \langle \Delta_{t_{\text{last}}} | \Delta x(t) \rangle + \frac{1}{2} \langle \Delta x(t) | \Gamma_{t_{\text{last}}} | \Delta x(t) \rangle \quad (25)$$

9.2 Properties

1. **$O(1)$ per unit.** Multiply cached Greeks by market moves.
2. **Accuracy degrades** with time and large moves. Self-correcting on next full reprice.
3. **Quality = APPROXIMATE.** Distinguished from FIRM and STALE.
4. **FSM context.** Available only in state EXPLAINED. When the unit transitions to STALE, the approximation is no longer trusted.

9.3 Official PnL Uses FIRM Prices Only

Principle 9.2 (FIRM-Only PnL). *The official PnL computation uses only FIRM Valuation-Records at configured valuation points. APPROXIMATE prices are acceptable for intraday risk monitoring but not for official PnL, regulatory reports, or end-of-day flash.*

10 The Valuation Store

ValuationRecords are stored in a valuation store indexed by (`unit_id`, `timestamp`, `model_id`). The store is append-only within a pricing cycle.

Five consumers: (1) Ledger PnL computation (reads `dirty_price`, FIRM only), (2) obligation-liveness executor (CSA margin), (3) risk pipeline (Greeks for limits), (4) PnL explain (reads previous FIRM record), (5) approximation service (reads latest FIRM record with Greeks).

Point-in-time queries support time travel: `snapshot_at(t)` returns the latest record per unit with `timestamp ≤ t`.

11 Edge Cases

11.1 Market Data Gap

Market data unavailability: downstream workflows wait or proceed with stale data. FSM: units depending on stale market data transition T8 to STALE.

11.2 Model Failure

Pricer crash, timeout, or NaN: FSM transition T3 to FAILED, then T10 retry with backoff. Fallback model hierarchy per instrument class.

11.3 DAG Mutation Mid-Cycle

New trades extend the DAG on the next cycle; terminated units are removed.

11.4 Monte Carlo Determinism

MC pricers use deterministic seeds: `seed = hash(market_data_snap, unit_id)`. Same inputs produce same outputs, satisfying purity [1, §7.7].

11.5 Taylor Approximation During Large Moves

Large market moves ($> 5\%$ since last FIRM): trigger immediate full reprice for affected units; APPROXIMATE prices flagged with “large move” warning; risk pipeline applies prudential haircut.

12 Worked Example: Complete Valuation Cycle

12.1 Setup

Three-instrument portfolio: NVDA spot (cadence 5s), NVDA Jun 2026 900 Call (cadence 30s), structured note embedding the call (cadence 5min).

Previous valuations (14:29:30 UTC):

Unit	Price	δ	γ	ν	θ	FSM state
NVDA	\$908.00	1.000	—	—	—	EXPLAINED
NVDA-C-Jun26-900	\$68.20	0.571	0.0039	0.298	-0.138	EXPLAINED
SN-NVDA-001	\$102.30	—	—	—	—	EXPLAINED

12.2 14:29:45 UTC — Approximate Pricing (FSM: EXPLAINED)

NVDA spot: \$910.20 ($\Delta S = +\2.20). Vol: 42.1% ($\Delta\sigma = +0.3\%$).

$$\begin{aligned}
 P_{\text{approx}}(\text{NVDA-C}) &= 68.20 + 0.571 \times 2.20 + \frac{1}{2} \times 0.0039 \times 2.20^2 + 0.298 \times 0.3 - 0.138 \times \frac{15}{365} \\
 &= \$69.55 \quad (\text{quality} = \text{APPROXIMATE}, \text{FSM: EXPLAINED})
 \end{aligned}$$

12.3 14:30:00 UTC — Full Reprice Cycle

Fresh data: NVDA \$912.50, vol 42.3%, rate 5.20%. All units transition through the FSM:

NVDA spot. FSM: EXPLAINED $\xrightarrow{T1/T9}$ PRICING $\xrightarrow{T2}$ PRICED $\xrightarrow{T4}$ EXPLAINING. PnL explain: $|912.50 - 908.00 - 1.0 \times 4.50| = 0$. PASS. $\xrightarrow{T5}$ EXPLAINED.

NVDA Call. BSM output: \$72.85, $\delta = 0.583$, $\gamma = 0.0041$. FSM: EXPLAINED $\rightarrow$ PRICING $\rightarrow$ PRICED $\rightarrow$ EXPLAINING. Explained PnL = \$4.159; actual = \$4.65; unexplained = \$0.49 < \$1.00 tolerance. PASS. $\rightarrow$ EXPLAINED.

Structured note. $P = 100 \times 72.85/71.00 = \102.61 . Unexplained $\approx$ \$0.00. PASS. $\rightarrow$ EXPLAINED.

12.4 Portfolio Valuation

$$V_{14:30:00} = 100 \times 912.50 + 50 \times 72.85 + 200 \times 102.61 = \$115,414.50$$

$$\text{PnL} = 115,414.50 - 114,670.00 = +\$744.50$$

Path-independent per [1, §4.3].

13 Consistency with the Lifecycle Engine

13.1 The Price Function Depends on Unit State

Principle 13.1 (State-Dependent Pricing). *The price function is:*

$$P_t(u) = P(u, \text{state}_t(u), \text{market_data}_t) \quad (26)$$

where $\text{state}_t(u)$ is the unit state dictionary from the lifecycle engine at time t . A pricer invocation without the current unit state is ill-defined.

This is the formal definition of the price function. The scalar compatibility principle (Principle 3.2) is preserved: `dirty_price` is still the $P_t(u)$ consumed by the PnL computation.

13.2 The Lifecycle-Valuation Ordering Protocol

Principle 13.2 (Lifecycle-Before-Valuation). *For any unit u , all pending lifecycle events must be processed before the next valuation cycle begins. The FSM transition T1/T9 (entering PRICING) requires:*

- (a) *fresh market data for all upstream dependencies, AND*
- (b) *confirmation that no pending lifecycle events exist for u .*

This is implemented via Temporal signals: the lifecycle workflow signals the pricing workflow after each event. The pricing workflow drains all pending lifecycle signals before reading unit state.

13.3 Lifecycle Events and the FSM

Material lifecycle events (Definition 13.3) force an FSM transition:

- If in EXPLAINED: transition to PRICING (bypass T8, immediate reprice).
- If in STALE: transition to PRICING (lifecycle event provides the trigger for T9).
- Cached Greeks are invalidated; Taylor approximation returns STALE until reprice completes.

Definition 13.3 (Repricing Classification). *A lifecycle event is **material** if it changes $P(u, \text{state}_t(u), \text{market_data}_t)$ for fixed market data. Material events: coupon payment, exercise, expiry, barrier breach, settlement, partial return. Non-material events: margin calls, collateral substitution, reporting.*

13.4 Taylor Approximation Invalidation

Lifecycle events create price discontinuities not captured by Greeks:

Event	Discontinuity
Coupon payment	Dirty price drops by coupon amount
Ex-dividend	Equity price drops by dividend amount
Option exercise/expiry	Price $\rightarrow$ intrinsic or 0
Barrier knock-in/out	Price jumps

On material lifecycle event: invalidate Greeks cache, return `quality = STALE` until reprice, optionally apply closed-form adjustment ($P_{\text{adj}} = P_{\text{last}} - \text{coupon}$) with `quality = APPROXIMATE`.

13.5 Extended Pricing DAG

The extended DAG $G' = (N, E_M \cup E_S)$ distinguishes market data edges (E_M) from state dependency edges (E_S). State edges are self-loops (unit state to unit pricer) and do not affect topological order.

13.6 Worked Example: Bond Coupon Payment

Setup. Corporate bond, 5% annual coupon, semi-annual \$2.50. Pre-coupon FIRM: dirty \$102.50, clean \$100.00, accrued \$2.50.

Sequence.

1. Lifecycle event fires (16:05): coupon paid, accrued resets. Signal sent to pricing workflow.
2. Taylor invalidated: approximate price = $\$102.50 - \$2.50 = \$100.00$ (APPROXIMATE).
3. Pricing workflow fires (16:06): post-coupon state, yield +2 bp. New dirty price \$99.93.
4. PnL explain: total $-\$2.57 =$ coupon $(-\$2.50) +$ theta $(+\$0.014) +$ duration $(-\$0.084)$. Unexplained \$0.00. PASS. FSM $\rightarrow$ EXPLAINED.

14 Production Scalability Analysis

14.1 Workflow Execution Volume

The one-workflow-per-unit design creates $|\mathcal{U}|$ concurrent Temporal workflows. The workflow execution rate depends on the instrument mix and cadence:

Instrument class	Count	Cadence	Cycles/min	Executions/min
Equity spot	10,000	5s	12	120,000
Vanilla options	50,000	30s	2	100,000
Exotic options	5,000	5min	0.2	1,000
IRS / bonds	20,000	1min	1	20,000
Structured notes	5,000	10min	0.1	500
Calibration workflows	100	10s	6	600
Total	90,100			$\approx 242,100$

At 100,000 units with a weighted-average cadence of approximately 25 seconds, the system generates approximately 240,000 workflow task completions per minute, or approximately 4,000 per second.

14.2 Temporal Cluster Sizing

Each pricing cycle involves 3–5 Temporal activities (market data retrieval, base pricing, Greek computation fan-out, PnL explain, publication). At 240K cycles/min with 4 activities each, the cluster processes approximately 16,000 activities per second.

Workflow workers (pricing-workflow). Each workflow is lightweight (state machine logic, signal handling). A single worker can handle 1,000–5,000 concurrent workflows. At 90,000 active workflows: 20–90 workflow workers.

Activity workers. Activity compute varies by queue:

- **pricing-fast:** 120K activities/min, < 1ms each. 2–4 workers.
- **pricing-analytical:** 100K activities/min, 1–10ms each. 10–20 workers.
- **pricing-mc:** 1,000 activities/min, 10–30s each. 50–100 GPU workers.
- **pricing-greeks:** proportional to analytical + MC. 20–50 workers.
- **pricing-calibration:** 600 activities/min. 2–4 workers.

Temporal server. The Temporal server cluster (frontend, history, matching services) must handle 16K activities/second dispatch rate. A horizontally scaled Temporal deployment with 3–5 history shards and Cassandra/MySQL backend handles this comfortably—Temporal’s published benchmarks show 10K–50K activities/second per shard.

14.3 History Management

Long-running pricing workflows (lifetime = unit lifetime, potentially years) accumulate event history. The `ContinueAsNew` pattern [1, §14.12] resets history every N cycles (configurable, e.g., 1,000 cycles or 24 hours), carrying forward only the FSM state, previous FIRM record, and freshness map. This bounds per-workflow history to a manageable size.

14.4 Scalability Summary

Metric	Value at 100K units
Concurrent workflows	~90,000
Workflow executions / minute	~240,000
Activity dispatches / second	~16,000
Workflow workers	20–90
Activity workers (total)	85–180
Temporal server shards	3–5
Peak memory (workflow state)	~9 GB (100 KB/workflow)

The architecture scales horizontally: doubling the unit count doubles the worker pool but does not change the per-unit resource consumption. The bottleneck is MC pricer compute for exotic options, addressed by GPU worker pools and cadence management (longer cadence for less liquid exotics).

15 Interaction Summary

Component	Interface	Data flow
Move stream [1, §8]	Read-only	Valuation store reads wallet balances for V_t .
Unit Store [1, §3]	Read-only	DAG constructed from <code>pricing_dependencies()</code> .
Calibration engine	Bidirectional	Kalman filter reads raw quotes, produces certified params for DAG leaf nodes.
Obligation-liveness [1, §14.7]	Write (alerts)	PnL explain failures and staleness create obligations.
Temporal [1, §14]	Bidirectional	PricingWorkflows, calibration workflows, Greek activities.
Lifecycle engine [1, §7]	Read + signals	Unit state for pricing; lifecycle signals for ordering and invalidation.

16 Conclusion and Open Problems

This document specifies the valuation layer that produces the price function P_t consumed by the Ledger framework. Version 1.0 makes five contributions beyond v0.2:

1. **Valuation lifecycle FSM** (Section 2): eight states, twelve transitions, every state defined, every transition guarded. The FSM makes explicit the progression from market data arrival through pricing, verification, and publication.
2. **Kalman filter for market data calibration** (Section 5): the full predict-update cycle with innovation-based gating, no-arbitrage constraints, and integration with the Pricing DAG as calibration nodes.
3. **Greeks, parameters, and the sensitivity Jacobian** (Section 4): a pricing model has parameters assumed to be martingales; Greeks are partial derivatives holding parameters constant. The scalar “vega” is shown to vanish for multi-parameter models, replaced by the sensitivity Jacobian—a vector in $\mathbb{R}^n$ where n is the parameter count. PnL attribution with the full Jacobian explicitly decomposes the “parameter PnL” that was previously lumped into the unexplained residual. Black–Scholes vs. local vol delta worked concretely, total gamma with vanna/volga/vega-curvature, the toxic product boundary, the consensus protocol for multi-model disagreement, and practical computation (analytical, bump-and-revalue, AAD, pathwise/likelihood ratio).
4. **Formal price function** (Section 13): $P(u, \text{state}_t(u), \text{market_data}_t)$ as a formal definition, lifecycle events as FSM transitions, Taylor approximation positioned in the FSM.
5. **Production scalability analysis** (Section 14): 100K units at mixed cadence yields $\sim 240\text{K}$ workflow executions/minute, within Temporal cluster capacity with 90–180 workers and 3–5 server shards.

16.1 Open Problems

Intraday curve bootstrapping. Should bootstrapping be a Temporal workflow in the Pricing DAG? The Kalman filter (Section 5) provides a natural answer: yes, as a calibration node. The open question is whether the bootstrapping algorithm itself (cubic spline, piecewise linear) should be inside the Kalman state or outside it.

Cross-asset correlation. PnL explain operates per-unit. Portfolio-level attribution requires cross-asset Greeks. The joint Kalman filter (Section 5) addresses the calibration side; extending to portfolio-level PnL explain is future work.

Model risk framework. The multi-model consensus (Section 4.11) specifies the operational protocol. Formal model risk reserve calculations, dual-valuation regulatory requirements, and automated model comparison are future work.

Adaptive cadence. The fixed cadence per instrument class (Section 7) is a simplification. Instruments approaching barriers, near expiry, or with rapidly changing Greeks need shorter cadences. Formal error bounds on the Taylor approximation and adaptive cadence selection are open.

Scaling beyond 1M units. The scalability analysis (Section 14) covers 100K units. Scaling to millions of listed derivatives requires workflow batching (multiple units per workflow), hierarchical DAG evaluation, and potentially moving from one-workflow-per-unit to one-workflow-per-portfolio-slice.

References

- [1] R. Delloye, “The Ledger: A Closed-System Framework for Financial Settlement—Unified Design Specification: Architecture, Lifecycle, and CDM Integration,” v10.3, March 2026.
- [2] R. Delloye, “On the Consistency of Derivative P&L Across Sequential Calculations Using Polynomial Approximations,” 2026.
- [3] R. Delloye, “P&L Attribution and Total Greeks: A Unified Framework via Polynomial Approximation and Smile-Adjusted Sensitivities,” 2026.
- [4] R. Delloye, “Gamma Adjustment with Quadratic Volatility: Final Derivation,” 2026.
- [5] R. Delloye, “Market Data Calibration Manifesto,” March 2026.
- [6] R. Delloye, “Modeling Implied Volatility via Kernel-Based Representations of the Curve,” 2026.
- [7] R. Delloye, “Implied and Local Volatility via Kernel-Based Curve Representations,” 2026.
- [8] R. Delloye, “Local Volatility in the Context of Quadratic Implied Volatility in Log-Moneyness,” 2026.
- [9] R. Delloye, “Pricing Complex Derivatives via Regression-Based Monte Carlo and Surrogate Models,” 2026.